Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

Звіт про виконання лабораторної роботи №4

«Об'єктно-орієнтована модель предметної області»

з курсу «Професійний Python»

Виконала:

студентка 3 курсу групи ФеП-32

Братівник Анастасія

Викладач: доцент Куньо І. М.

Львів – 2026

Мета: Сформувати практичні навички побудови професійної об'єктно-орієнтованої моделі в Python із використанням класів, композиції, наслідування, абстрактних базових класів, протоколів, dataclass, properties, dunder methods, принципів SOLID та сучасної системи type hints.

Індивідуальний варіант і постановка задачі

Варіант №1 — «Система обліку студентів».

Domain objects: Student, Group, AcademicRecord, Grade, StudentRegistry.

Реалізовано:

- додавання та редагування даних студента;
- пошук за прізвищем та фільтрація за групою;
- обчислення середнього бала групи;
- визначення найкращого студента;
- generic repository;
- reporting service;
- notification abstraction.

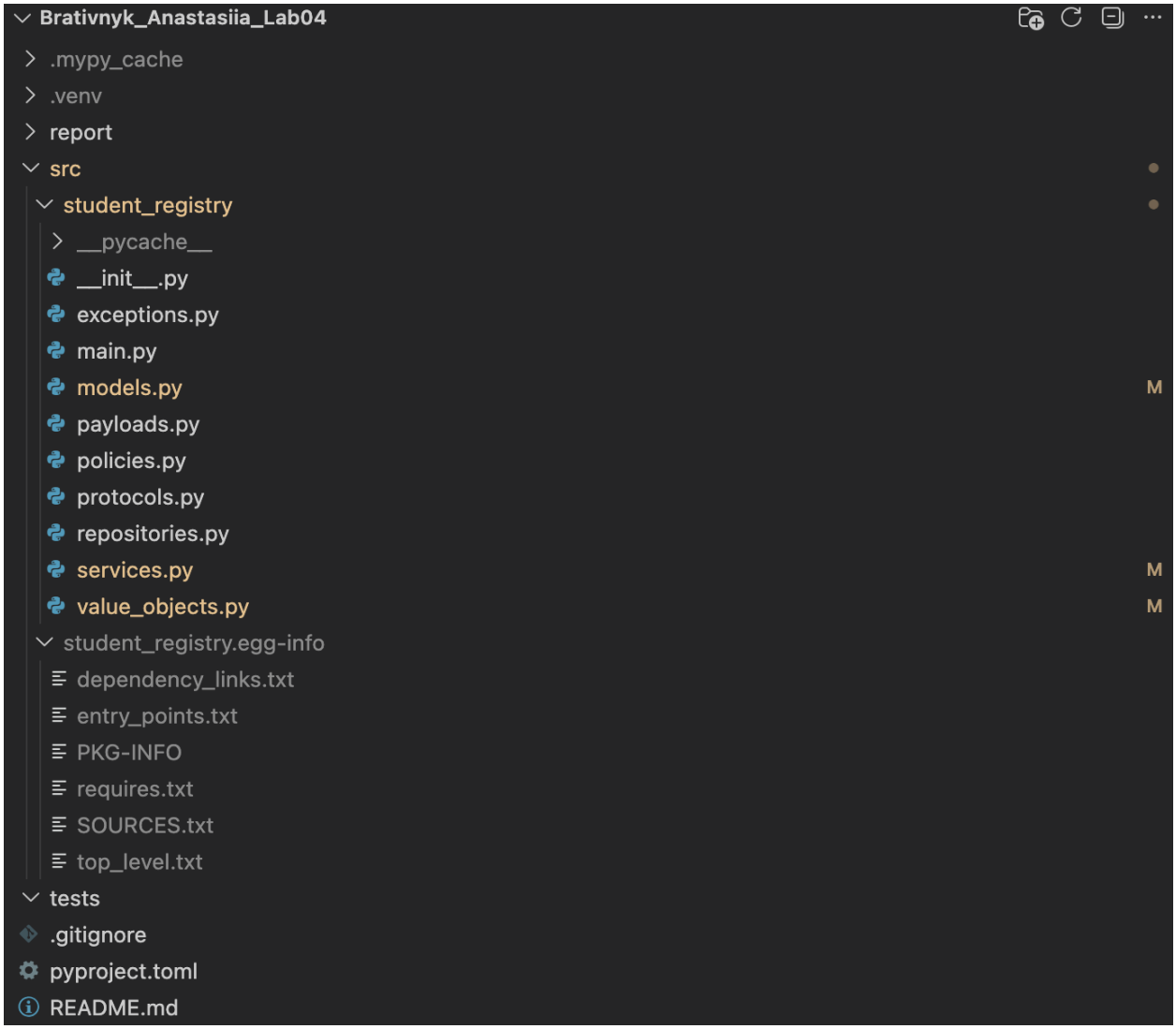
Опис предметної області

Предметна область — облік студентів університету. Студент належить до групи та має середній бал (Grade). Бал — це immutable value object із діапазоном [0, 100]. Група — агрегат, що містить student_ids. AcademicRecord зв'язує студента з дисципліною та оцінкою. StudentRegistryService координує всі use cases.

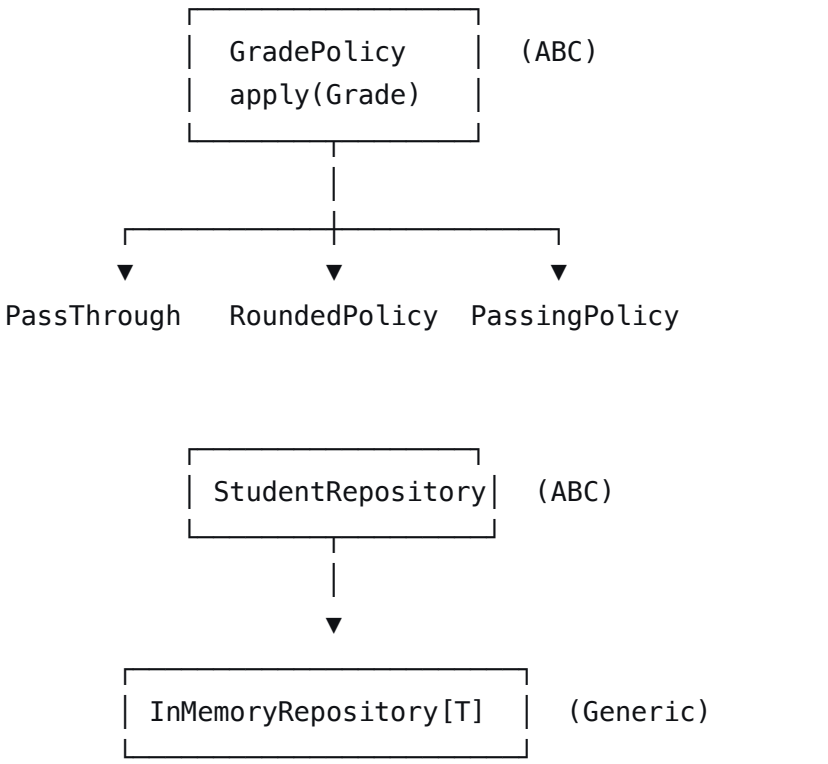
Список domain entities

Клас	Роль	Тип
Student	Domain entity	dataclass
Group	Aggregate	dataclass
AcademicRecord	Domain entity	dataclass
Grade	Value object	frozen dataclass
StudentRegistryService	Application service	class
InMemoryRepository[T]	Infrastructure	Generic class
GradePolicy (ABC)	Policy abstraction	ABC
Notifier, Exporter, HasId	Protocols	Protocol

Структура проєкту



Class diagram



Пояснення inheritance

Наслідування використано для GradePolicy → PassThroughPolicy / RoundedPolicy / PassingPolicy. Це **is-a** відношення: кожна конкретна policy **є** GradePolicy. Дозволяє додавати нові політики без зміни StudentRegistryService (Open/Closed Principle).

Пояснення composition

StudentRegistryService **has-a** InMemoryRepository[Student], Notifier, Exporter, GradePolicy. Це composition — сервіс використовує ці об'єкти як частини реалізації. Дозволяє тестувати та замінювати залежності без зміни сервісу.

Student **has-a** Grade — композиція value object.

Приклад encapsulation

Приватні атрибути: Student._average_grade, Group._student_ids, InMemoryRepository._items. Зовнішній код не може їх змінити напряму — тільки через методи update_grade, add_student, add.

Приклад property

```
@property
def full_name(self) -> str:
    return f"{self.first_name} {self.last_name}"


@average_grade.setter
def average_grade(self, value: Grade) -> None:
    self._average_grade = value
```

Приклад dataclass

Student, Group, AcademicRecord — звичайні dataclass з __post_init__ для валідації. Grade — frozen dataclass.

Приклад immutable value object

```
@dataclass(frozen=True, slots=True)
class Grade:
    value: float

    def __post_init__(self) -> None:
        if not 0.0 <= self.value <= 100.0:
            raise ValueError(...)
```

Приклад ABC

```
class StudentRepository(ABC):
    @abstractmethod
    def add(self, student: Student) -> None: ...

    @abstractmethod
    def get(self, student_id: int) -> Student | None: ...
```

Створити StudentRepository() напряму не можна — потрібна конкретна реалізація.

Приклад Protocol

```
class Notifier(Protocol):
    def send(self, recipient: str, message: str) -> None: ...
```

ConsoleNotifier не успадковує Notifier явно — але завдяки structural typing (duck typing) підходить як аргумент.

Щонайменше 3 dunder methods

Метод	Клас	Призначення
__repr__	Student	Debug-представлення
__str__	Student, Grade	User-friendly вивід
__eq__	Student	Порівняння за student_id
__hash__	Student	Можна використовувати у set
__len__	Group, InMemoryRepository	len(group), len(repo)
__contains__	Group, InMemoryRepository	1 in group, 1 in repo
__iter__	InMemoryRepository	for s in repo
__add__	Grade	Середнє двох оцінок
__lt__	Grade	Порівняння оцінок

Generic class

```
T = TypeVar("T", bound=HasId)


class InMemoryRepository(Generic[T]):
    def __init__(self) -> None:
        self._items: dict[int, T] = {}
    def add(self, item: T) -> None: ...
```

TypeVar

T = TypeVar("T", bound=HasId) — обмежений TypeVar. Гарантує, що T має атрибут student_id.

TypedDict

class StudentPayload(TypedDict):

student_id: int

first_name: str

last_name: str

group: str

average_grade: float

email: NotRequired[str]

Використовується для даних із зовнішніх джерел (CSV/JSON). **Не є domain entity** — конвертується у Student через Student.from_payload().

Application service

StudentRegistryService — orchestration layer. Координує репозиторій, notifier, exporter, grade policy. Не містить бізнес-логіки — делегує її відповідним компонентам (SRP).

Dependency Injection

def __init__(

self,

repository: InMemoryRepository[Student],

notifier: Notifier,

exporter: Exporter,

grade_policy: GradePolicy,

) -> None:

...

Усі залежності передаються ззовні. Легко підмінити на mock для тестів.

SOLID

SRP — кожен клас має одну відповідальність: Student — дані студента, GradePolicy — обробка оцінок, Notifier — нотифікації, StudentRegistryService — orchestration.

OCP — нову політику оцінювання (PassingPolicy) додано без зміни StudentRegistryService. Просто передали іншу реалізацію GradePolicy.

DIP — StudentRegistryService залежить від абстракцій (Notifier, Exporter, GradePolicy, StudentRepository), а не від конкретних класів. Високорівнева логіка не знає про ConsoleNotifier чи CsvExporter.

Основний код програми

```
from dataclasses import dataclass


@dataclass(frozen=True, slots=True)
class Grade:
    """Immutable numeric grade with validation."""

    value: float

    def __post_init__(self) -> None:
        if not 0.0 <= self.value <= 100.0:
            raise ValueError(
                f"Grade must be in [0, 100], got {self.value}"
            )

    def __add__(self, other: "Grade") -> "Grade":
        return Grade((self.value + other.value) / 2)
```

```
    def __lt__(self, other: "Grade") -> bool:
        return self.value < other.value

    def __str__(self) -> str:
        return f"{self.value:.2f}"

    def __format__(self, format_spec: str) -> str:
        """Delegate format spec to the underlying float.

        Enables f-strings like f"{grade:.2f}" or f"{grade:>7}".
        """
        return format(self.value, format_spec)
```

```
from dataclasses import dataclass, field

from student_registry.exceptions import ValidationError
from student_registry.payloads import StudentPayload
from student_registry.value_objects import Grade


@dataclass
class Group:
    """Academic group — aggregate of students."""

    name: str
    _student_ids: list[int] = field(default_factory=list, init=False)

    def __post_init__(self) -> None:
        if not self.name.strip():
            raise ValidationError("Group name cannot be empty")

    def __len__(self) -> int:
        return len(self._student_ids)

    def __contains__(self, student_id: int) -> bool:
        return student_id in self._student_ids

    def add_student(self, student_id: int) -> None:
        if student_id in self._student_ids:
            return
        self._student_ids.append(student_id)

    def remove_student(self, student_id: int) -> None:
        if student_id in self._student_ids:
            self._student_ids.remove(student_id)

    @property
    def student_ids(self) -> tuple[int, ...]:
        return tuple(self._student_ids)


@dataclass
```

```
class Student:
    """Domain entity representing a student."""

    student_id: int
    first_name: str
    last_name: str
    group: str
    _average_grade: Grade

    def __post_init__(self) -> None:
        if not self.first_name.strip() or not self.last_name.strip():
            raise ValidationError("First and last name must not be empty")
        if not self.group.strip():
            raise ValidationError("Group must not be empty")

    @property
    def full_name(self) -> str:
        return f"{self.first_name} {self.last_name}"

    @property
    def average_grade(self) -> Grade:
        return self._average_grade

    @average_grade.setter
    def average_grade(self, value: Grade) -> None:
        self._average_grade = value

    def __repr__(self) -> str:
        return (
            f"Student(id={self.student_id}, "
            f"name={self.full_name!r}, "
            f"group={self.group!r}, "
            f"grade={self._average_grade})"
        )

    def __str__(self) -> str:
        return (
            f"{self.student_id:>4}  "
            f"{self.full_name:<25}"
            f"{self.group:<8}"
            f"{self._average_grade:>7.2f}"
        )

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Student):
            return NotImplemented
        return self.student_id == other.student_id

    def __hash__(self) -> int:
        return hash(self.student_id)

    @classmethod
    def from_payload(cls, payload: StudentPayload) -> "Student":
```

```
        return cls(
            student_id=payload["student_id"],
            first_name=payload["first_name"],
            last_name=payload["last_name"],
            group=payload["group"],
            _average_grade=Grade(payload["average_grade"]),
        )


@dataclass
class AcademicRecord:
    """Associates a student with a discipline and a grade."""

    student_id: int
    discipline: str
    grade: Grade

    def __post_init__(self) -> None:
        if not self.discipline.strip():
            raise ValidationError("Discipline must not be empty")
```

```
from collections.abc import Iterable

from student_registry.exceptions import NotFoundError, ValidationError
from student_registry.models import Group, Student
from student_registry.policies import GradePolicy
from student_registry.protocols import Exporter, Notifier
from student_registry.repositories import InMemoryRepository
from student_registry.value_objects import Grade


class StudentRegistryService:
    """Orchestrates use cases around students."""

    def __init__(
        self,
        repository: InMemoryRepository[Student],
        notifier: Notifier,
        exporter: Exporter,
        grade_policy: GradePolicy,
    ) -> None:
        # Dependency Injection через конструктор
        self._repo = repository
        self._notifier = notifier
        self._exporter = exporter
        self._policy = grade_policy

    # --- commands ---

    def register_student(self, student: Student) -> Student:
        adjusted = Student(
            student_id=student.student_id,
```

```
            first_name=student.first_name,
            last_name=student.last_name,
            group=student.group,
            _average_grade=self._policy.apply(student.average_grade),
        )
        self._repo.add(adjusted)
        self._notifier.send(
            "registry@university",
            f"Registered: {adjusted.full_name}",
        )
        return adjusted

    def update_grade(self, student_id: int, new_grade: Grade) -> Student:
        student = self._repo.require(student_id)
        student.average_grade = self._policy.apply(new_grade)
        return student

    def remove_student(self, student_id: int) -> bool:
        return self._repo.remove(student_id)

    # --- queries ---

    def find_by_last_name(self, last_name: str) -> list[Student]:
        needle = last_name.strip().lower()
        return [
            s for s in self._repo.all()
            if s.last_name.lower() == needle
        ]

    def filter_by_group(self, group: str) -> list[Student]:
        return [s for s in self._repo.all() if s.group == group]

    def best_student(self) -> Student | None:
        students = self._repo.all()
        if not students:
            return None
        return max(students, key=lambda s: s.average_grade)

    def average_of_group(self, group: str) -> float:
        students = self.filter_by_group(group)
        if not students:
            return 0.0
        total = sum(s.average_grade.value for s in students)
        return total / len(students)

    def export_all(self) -> str:
        return self._exporter.export(self._repo.all())

    def build_group(self, name: str) -> Group:
        group = Group(name=name)
        for student in self.filter_by_group(name):
            group.add_student(student.student_id)
        return group
```

Screenshot execution

```
(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab04 % python -m student_registry.main
================================================================
LAB 04 — OOP domain model (Variant 1: Student registry)
================================================================

=== REGISTERING STUDENTS ===
[notify -> registry@university] Registered: Anna Koval
[notify -> registry@university] Registered: Oleh Melnyk
[notify -> registry@university] Registered: Iryna Bondar
[notify -> registry@university] Registered: Andrii Shevchenko
[notify -> registry@university] Registered: Maria Boyko

=== ALL STUDENTS ===
   1  Anna Koval               PI-21     92.00
   2  Oleh Melnyk              PI-21     84.00
   3  Iryna Bondar             PI-22     95.00
   4  Andrii Shevchenko        PI-21     89.00
   5  Maria Boyko              PI-22     79.00

=== SEARCH: 'Melnyk' ===
   2  Oleh Melnyk              PI-21     84.00

=== GROUP PI-21 ===
   1  Anna Koval               PI-21     92.00
   2  Oleh Melnyk              PI-21     84.00
   4  Andrii Shevchenko        PI-21     89.00

=== AVERAGE GRADES ===
  PI-21: 88.33
  PI-22: 87.00

Best student: Iryna Bondar (95)

=== UPDATE GRADE (id=2) ===
Updated:    2  Oleh Melnyk              PI-21     93.00

Group PI-21: 3 students
Contains id=1? True

=== CSV EXPORT ===
student_id,name,group,grade
1,Anna Koval,PI-21,92
2,Oleh Melnyk,PI-21,93
3,Iryna Bondar,PI-22,95
4,Andrii Shevchenko,PI-21,89
5,Maria Boyko,PI-22,79
(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab04 %
```

Результат mypy

До виправлення (з навмисними помилками):

```
(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab04 % mypy src

  Success: no issues found in 10 source files
(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab04 % mypy src/student_registry/type_errors_demo.py
  src/student_registry/type_errors_demo.py:8: error: Incompatible types in assignment (expression has type "str", variable has type "int")  [assignment]
  src/student_registry/type_errors_demo.py:12: error: Incompatible return value type (got "str", expected "int")  [return-value]
  src/student_registry/type_errors_demo.py:16: error: Argument 1 to "add" of "InMemoryRepository" has incompatible type "str"; expected "Student"  [arg-type]
  src/student_registry/type_errors_demo.py:17: error: Too many arguments for "add" of "InMemoryRepository"  [call-arg]
  src/student_registry/type_errors_demo.py:24: error: Property "value" defined in "Grade" is read-only  [misc]
  Found 5 errors in 1 file (checked 1 source file)
(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab04 %
```

Після виправлення:

```
(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab04 % rm src/student_registry/type_errors_demo.py

(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab04 % mypy src

  Success: no issues found in 10 source files
(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab04 %
```

Приклади знайдених type errors

#	Помилка	Виправлення
1	student_id: int = "1"	student_id: int = 1
2	def total() -> int: return "100"	повертати int
3	repo.add("not a student")	передати Student
4	grade.value = 100.0 (frozen)	не мутувати, створити новий Grade(100.0)

Робота з Git

```
(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab04 % git log --oneline

0fbedcb (HEAD -> master) Add application service, DI and entry point
963e147 Add protocols, ABC and generic repository
3c6ce0b Add domain models and grade policies
0e0b291 Add exceptions, value objects and TypedDict payloads
b73b1a5 Create project structure for Lab04
(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab04 % git status

On branch master
nothing to commit, working tree clean
```

Репозиторій на GitHub

https://github.com/BtativnykA/Brativnyk_Anastasiia_Lab04

Висновок:

У лабораторній роботі №4 побудовано типізовану об'єктну модель предметної області «Система обліку студентів». Створено 5 domain classes (Student, Group, AcademicRecord, Grade, InMemoryRepository), 2 dataclass + 1 frozen value object (Grade), властивість із setter (Student.average_grade), наслідування (GradePolicy → 3 concrete policies), композицію (Student has-a Grade, Service has-a Repository), ABC (StudentRepository, GradePolicy), 3 Protocol (Notifier, Exporter, HasId), 9 dunder methods (__repr__, __str__, __eq__, __hash__, __len__, __contains__, __iter__, __add__, __lt__), Generic repository з обмеженим TypeVar, TypedDict для зовнішніх payloads, DI через конструктор сервісу. Реалізовано принципи SOLID (SRP, OCP, DIP). Код перевірено mypy --strict:

спочатку навмисно внесено 5 помилок — усі виявлено checker'ом; після виправлення — Success: no issues found in 10 source files.

Отримано фундамент для подальшої роботи: тестування через підміну Protocol-залежностей, інтеграція з БД через реалізацію StudentRepository, розширення політик оцінювання без зміни сервісу.